

ATHENS UNIVERSITY OF ECONOMICS AND BUSINESS

Instructor: Ion Androutsopoulos

Assignment 1 - n-gram Language Models

1. Theofanopoulos Michail, p3352401 , mic.theofanopoulos@aueb.gr
2. Mantzaris Marios, p3352421 , mar.mantzaris@aueb.gr
3. Kitsakis Georgios, p3352406 , geo.kitsakis@aueb.gr
4. Zacharis Efstathios, p3352415 , efs.zacharis@aueb.gr

The link to the Colab notebook is: [Link](#)

0. Project Scope.....	3
1. Bigram and Trigram Language Models.....	3
1.1 N-Gram Language Model class.....	3
1.2 Data Preprocessing.....	4
1.3 Data Loading and Splitting.....	4
2. Cross Entropy and Perplexity.....	4
2.1 Methods.....	4
2.2 Results.....	5
3. Auto Text Completion.....	6
3.1 Methods.....	6
generate_text.....	6
get_next_word_greedy.....	6
get_next_word_topk.....	7
beam_search.....	7
3.2 Prompts Results.....	8
4. Spelling Error Corrector.....	10
4.1 Methods.....	10
ContextAwareSpellingCorrector.....	10
log_prob.....	10
calculate_lm_score.....	10
calculate_error_score.....	11
combine_scores.....	11
generate_candidates.....	11
beam_search_step.....	11
correct.....	11
4.2 Results.....	12
5. Artificial Test Dataset.....	14
5.1 Overview.....	15
5.2 Implementation.....	15
Detailed Explanation.....	15
5.3 Sample Output.....	15
5.4 Results.....	16
6. Spelling Corrector Evaluation.....	16
6.1 Overview.....	16
6.2 Implementation.....	17
6.3 Results.....	17
Interpretation.....	18
6.4 Conclusion.....	18

0. Project Scope

In this project, we implemented n-gram language models and a spelling corrector system trained and evaluated on Brown corpus, available from [NLTK](https://www.nltk.org/book/ch02.html) library. According to the library's documentation (<https://www.nltk.org/book/ch02.html>) Brown corpus contains a collection of, multi-source, text samples of American English that come from 500 sources. Regarding the overall structure of the python script, we followed an object-oriented approach with classes defining the main objects used, while custom functions perform specific tasks.

1. Bigram and Trigram Language Models

1.1 N-Gram Language Model class

In order to create the N-gram language model we created a class called ``NgramLanguageModel`` with the following methods (The class has more methods that are useful for part 2, we will describe them later)

`__init__`

Initializes the N-gram language model with parameters like n-gram size, minimum word frequency, tokenizer type, and sets up model components like vocabulary, counters, and special tokens.

`custom_tokenize(text)`

Tokenizes a text into lowercase word tokens using a custom regular expression-based approach.

`preprocess_text(corpus)`

Converts a list of text passages into tokenized sentences by splitting into sentences and then applying the chosen tokenizer.

`build_vocabulary(tokenized_sentences)`

Builds the model's vocabulary by selecting words that meet the minimum frequency requirement and adding special tokens.

`replace_oov_words(tokenized_sentences)`

Replaces words not found in the vocabulary with a special `<UNK>` token across all tokenized sentences.

`extract_ngrams(tokenized_sentences)`

Adds start and end tokens, extracts n-grams from sentences, and updates counts for both n-grams and their contexts.

train(corpus)

Full pipeline to train the model: preprocesses the corpus, builds the vocabulary, replaces OOV words, extracts n-grams, and logs training statistics. We used the **brown** corpus

1.2 Data Preprocessing

The main pain point of creating the model was to figure out which tokenizer to use. We considered many alternatives, including spacy. Some of the alternatives showed good results but the time it took to generate the model was way too long, so we opted to not use them. In the end, we had 3 choices

1. **NLTK's word_tokenize**: A comprehensive tokenizer that handles punctuation as separate tokens and properly manages contractions.
2. **Custom regex tokenizer** (`\b\w+\b`): A simplified approach that captures only alphanumeric characters and underscores while completely ignoring punctuation.
3. **RegexpTokenizer** with pattern `\w+|'(|)\|\.|,`: A more selective approach that captures words plus specific punctuation marks as individual tokens.

We selected the NLTK tokenizer as it seemed the best choice for the rest of the project needs and applications (although the other tokenizers would yield similar results in most cases)

1.3 Data Loading and Splitting

The `load_and_split_corpus` function is responsible for loading the specified corpus (Brown) from the NLTK library, preprocessing it into sentences, and partitioning it into training, validation, and testing sets.

2. Cross Entropy and Perplexity

2.1 Methods

Methods inside the class

```
get_laplace_probability(word, context, alpha=0.01)
```

Calculates the conditional probability $P(\text{word}|\text{context})$ using **Laplace (additive) smoothing** to handle unseen n-grams.

Alpha is a smoothing parameter, the purpose of this parameter is to:

1. Prevent zero probabilities for n-grams that were never seen in the training data
2. Distribute some probability mass to unseen events

3. Make the model more robust to rare or unseen words/sequences

Higher alpha values give more probability mass to unseen or rare sequences. This makes the model more robust when dealing with small training samples or lots of unseen words.

Lower alpha values give less probability to unseen events and trust the observed counts more. This works better when you have large training samples with good coverage of the vocabulary and common word sequences.

get_log_probability(word, context)

Returns the **logarithm (base 2)** of the probability for a given word and context; useful for numerical stability and sentence scoring.

get_sentence_log_probability(sentence)

Computes the **total log probability** of an entire sentence by summing the log probabilities of each word given its preceding context (with OOV words handled).

- We also have methods that are not part of the class

calculate_cross_entropy(model, test_corpus)

Computes the **cross-entropy** of a trained language model on a test corpus, measuring the model's average uncertainty.

calculate_perplexity(cross_entropy)

Converts a **cross-entropy score** into **perplexity**, a common metric that indicates how well the language model predicts the test data (lower is better).

2.2 Results

Results:

VALIDATION SET	Cross-Entropy	Perplexity
Bigram	7.32	160.6
Trigram	9.2	590.31

TEST SET	Cross-Entropy	Perplexity
Bigram	7.3	157.88
Trigram	9.16	574.35

At a glance, the above results do not make sense, since, in theory, a trigram model should be better than a bigram model, so the cross-entropy of the trigram should be lower. But in practice this may happen due to:

- Data sparsity: Trigrams need way more data to estimate good probabilities. The reason is that we have way fewer combinations of (w_1, w_2, w_3) than (w_1, w_2)
- Small vocabulary: Sharing a vocabulary, which is also relatively very small, may lead to worse performance of trigrams when compared to bigrams

3. Auto Text Completion

3.1 Methods

`generate_text`

Generates a text continuation given a prompt using one of two decoding methods: greedy, top-k sampling

`get_next_word_greedy`

Returns the most probable next word given the context by picking the highest probability.

- Greedy decoding simply chooses the word with the max probability at each time step.
- Produces very *predictable* but less diverse outputs.

How it works:

1. Calculates the probability for every word in the vocabulary following the current context
2. Excludes the `UNK` token from consideration (we don't want to generate unknown words)
3. Selects the single word with the highest probability

`get_next_word_topk`

Samples the next word from the top-k most probable words (randomly, based on probability).

- Allows controlled randomness to produce more diverse and creative text.
- Temperature helps:
 - Low temp (0.5) = more conservative.
 - High temp (1.5) = more adventurous.

How it works:

1. Calculates the probability for every word in the vocabulary given the context
2. Ranks all words by probability and selects only the top K candidates (also skips **UNK** token)
3. Normalizes the probabilities of these K candidates to sum to 1
4. Applies temperature scaling to adjust the probability distribution:
 - Higher temperature (>1.0): Makes distribution more uniform (more random)
 - Lower temperature (<1.0): Makes distribution more peaked (less random)
5. Randomly samples the next word from this adjusted distribution

beam_search

Expands multiple potential completions (beams) and keeps the top-scoring ones.

- Keeps track of multiple partial completions instead of a single path (more global search).
- Beam width determines how many alternative completions to consider.
-

How it works:

1. Maintains multiple candidate sequences (beams) at each step
2. For each existing beam:
 - Calculates probabilities for all possible next words
 - Creates new candidate sequences by adding each word
 - Computes a score for each new sequence (sum of log probabilities)
3. Keeps only the top-scoring beams (determined by beam width)
4. Continues until reaching max length or all beams end
5. Returns all final beam sequences

3.2 Prompts Results

Some prompts that we used

```
prompts = [  
    "I would like to commend the",  
    "The president of",  
    "According to recent",  
    "In the last few",  
    "Experts say that"  
]
```

Some notable observations (while using NLTK tokenizer)

- Difference between bigram and trigram

When using **bigram** model

```
[Beam] I would like to commend the `` .
[Beam] The president of the `` .
[Top-K] Experts say that the `` .
```

We see that the using Beam method after finding `the` it is followed by `` and then period before ending (we can see the same in Top-K since there is a chance for Top-K to select it), so every time `the` token appears it will ALWAYS have [``.] after it but in the **trigram** model

```
[Beam] I would like to commend the world .
[Beam] The president of the united states .
```

Since now it has a wider context (e.g includes `commend the`) it yields different results every time it finds `the` depending on the context. The sentences also make a bit more sense.

- Repetitiveness of Greedy and Beam method in bigrams

```
[Greedy Bigram] In the last few days , and the same time , and the same time ,
and the same time , and the same
```

```
[Greedy Bigram] The president of the same time , and the same time , and the
same time , and the same time , and
```

Since greedy always chooses the most probable word, especially in the bigram, it can easily be stuck in a loop. So in bigrams, every time it finds either one of `days , and the same time` it will always repeat the same sentence. This also happens in the beam method.

- Degeneration

```
[Greedy Trigram] According to recent existence existence existence existence
existence existence existence existence existence existence existence existence
existence existence existence existence existence existence existence existence
[Top-K Trigram] According to recent recent blow recent existence blow blow blow
existence existence blow recent ways blow blow blow ways ways crystal ways blow
[Beam Trigram] According to recent existence existence existence existence
existence existence existence existence existence existence existence existence
existence existence existence existence existence existence existence existence
```

Trigrams can also be stuck in loops, that is called degeneration. We can see that both the greedy and beam methods get stuck only writing existence over and over. The Top-K tries to escape and has some randomness to it, but still repeats similar words

All of the completions

```
Bigram model completions
[Greedy] I would like to commend the same time , and the same time , and the
same time , and the same time , and the
[Top-K] I would like to commend the first , and a new and a little doubt .
[Beam] I would like to commend the `` .
```



```
[Greedy] Experts say that the united states , and the other hand , the `` public  
`` through a door handle `` , he  
[Top-K] Experts say that in this manner , and the same .  
[Beam] Experts say that it is not the only one of the united states .
```

4. Spelling Error Corrector

4.1 Methods

ContextAwareSpellingCorrector

In order to create the context aware spelling corrector we created a class called `ContextAwareSpellingCorrector` with the following methods:

`log_prob`

This method takes as a parameter the p-value and returns the log-probability, while safe handling zero-case.

`calculate_lm_score`

This method takes as a parameter a candidate and it's n-size context, and returns the model's log-probability.

`calculate_error_score`

This method takes as a parameter a noisy token and it's candidate and returns the log-probability using edit distance.

`combine_scores`

This method takes as a parameter the language model score, the error score, the lambdas and returns the score that a candidate will produce.

`generate_candidates`

This method generates candidates for a given noisy token using a given n-gram vocabulary. If skip out-of-vocabulary words option is enabled and the given token already exists in the vocabulary, we return a list containing the token.

`beam_search_step`

This method is our implementation of the beam_search algorithm. It iterates all beams, generates candidates for the given noisy token and calculate the score for each one. Finally it returns the top-n probable candidates.

correct

This method is the base method called when we want to correct a list containing noisy tokens.

4.2 Results

A sample run was executed using the above class. Some of the results are the following:

- For the **Bigram Model**:

```
Starting correction for sentence: Tomorrrrow well bring something new , so liv today as a memoory .
```

```
Token: 'Tomorrrrow' | Context: ('<start>',)
```

```
Top candidates:
```

Candidate	LM Score	Error Score	Combined
-----------	----------	-------------	----------

tomorrow	-15.0949	-1.0986	-12.2957
----------	----------	---------	----------

```
Best candidate selected: tomorrow
```

```
Token: 'well' (in vocabulary, skipping correction)
```

```
Token: 'bring' (in vocabulary, skipping correction)
```

```
Token: 'something' | Context: ('bring',)
```

```
Top candidates:
```

Candidate	LM Score	Error Score	Combined
-----------	----------	-------------	----------

something	-19.6206	-0.6931	-15.8351
-----------	----------	---------	----------

```
Best candidate selected: something
```

```
Token: 'new' (in vocabulary, skipping correction)
```

```
Token: ',' (in vocabulary, skipping correction)
```

```
Token: 'so' (in vocabulary, skipping correction)
```

```
Token: 'liv' | Context: ('so',)
```

```
Top candidates:
```

Candidate	LM Score	Error Score	Combined
-----------	----------	-------------	----------

i	-5.6515	-1.0986	-4.7409
---	---------	---------	---------

it	-5.9030	-1.0986	-4.9421
----	---------	---------	---------

in	-7.7103	-1.0986	-6.3880
----	---------	---------	---------

is	-7.9734	-1.0986	-6.5984
----	---------	---------	---------

did	-7.9734	-1.0986	-6.5984
-----	---------	---------	---------

```
Best candidate selected: i
```

```
Token: 'today' (in vocabulary, skipping correction)
```

```
Token: 'as' (in vocabulary, skipping correction)
```

```
Token: 'a' (in vocabulary, skipping correction)
```

```
Token: 'memoory' | Context: ('a',)
```

```
Top candidates:
```

Candidate	LM Score	Error Score	Combined
-----------	----------	-------------	----------

memory	-27.1004	-0.6931	-21.8189
--------	----------	---------	----------

memory	-27.1004	-0.6931	-21.8189
--------	----------	---------	----------

memory	-27.1004	-0.6931	-21.8189
--------	----------	---------	----------

memory	-27.1004	-0.6931	-21.8189
--------	----------	---------	----------

memory	-27.1004	-0.6931	-21.8189
--------	----------	---------	----------

```
Best candidate selected: memory
```

```
Token: '.' (in vocabulary, skipping correction)
```

```
Final corrected sentence: tomorrow well bring something new , so i today as a memory .
```

- For the **Trigram Model**:

```

Starting correction for sentence: let us sai we are freends

Token: 'let' (in vocabulary, skipping correction)

Token: 'us' (in vocabulary, skipping correction)

Token: 'sai' | Context: ('let', 'us')
Top candidates:
Candidate      LM Score  Error Score  Combined
-----
say            -4.2708    -0.6931     -3.5553
see            -4.2708    -1.0986     -3.6364
san            -18.5586   -0.6931     -14.9855
said           -18.5586   -0.6931     -14.9855
ai             -18.5586   -0.6931     -14.9855
Best candidate selected: say
-----

Token: 'we' (in vocabulary, skipping correction)

Token: 'are' (in vocabulary, skipping correction)

Token: 'freends' | Context: ('we', 'are')
Top candidates:
Candidate      LM Score  Error Score  Combined
-----
friends        -20.3168   -0.6931     -16.3921
friends        -20.3168   -0.6931     -16.3921
trends         -20.3168   -1.0986     -16.4732
freed          -20.3168   -1.0986     -16.4732
friend         -20.3168   -1.0986     -16.4732
Best candidate selected: friends
-----

Final corrected sentence: let us say we are friends

```

We can observe from the results that the **Trigram model** achieves significantly better corrections compared to the **Bigram model**. One key reason for this improvement is the **larger context window** available in the trigram model, which allows it to consider two preceding words instead of just one, leading to more accurate predictions.

Although the **edit distance** also influences the candidate selection, the **language model probability** has a much stronger impact on the final decision, especially because we use a $\lambda_{LM} = 0.8$. This high weight gives the language model a dominant role in ranking candidates, making the quality of the context modeling even more critical.

5. Artificial Test Dataset

Objective: Create an artificial test set by introducing character-level errors into the existing clean test corpus, to evaluate the context-aware spelling corrector under realistic typo conditions.

5.1 Overview

We generate a corrupted version of our test sentences by replacing each non-space character with a random substitute (letter, digit, or punctuation) at a fixed probability. This simulates typical typing mistakes (e.g., adjacent-key errors, substitution errors) without altering sentence structure or tokenization.

5.2 Implementation

Detailed Explanation

- **import random and import string**
 1. **random**: for stochastic choices in when and how to corrupt characters.
 2. **string**: to access predefined sets of characters (**ascii_letters**, **digits**, **punctuation**) for replacement.
- **Class Name: ArtificialTestDataset**
Encapsulates logic for generating a noisy test set from clean sentences.
- **__init__ method**
 1. **sentences**: the list of original, clean sentences (**List[str]**).
 2. **error_prob**: the probability with which each non-space character will be replaced (default 0.05 = 5%).
 3. **seed**: optional integer to seed the random number generator for reproducible outputs. If provided, calls **random.seed(seed)**.
 4. **self.chars**: builds the replacement pool by concatenating **string.ascii_letters**, **string.digits**, and **string.punctuation**, then casting to a list.
- **_corrupt_char(self, c)**
Responsible for deciding whether to corrupt a single character:
 1. If **c.isspace()** (i.e., it's a space, newline, or tab), or if **random.random()** (uniform in $[0,1)$) exceeds **error_prob**, return **c** unchanged.
 2. Otherwise, pick a random character **r** from **self.chars**.
 3. Use a **while** loop to ensure **r != c**, so we never "replace" a character with itself.
 4. Return the new character **r**.
- **generate(self)**
Produces the full corrupted dataset by:
 1. Iterating over each sentence in **self.sentences**.
 2. Applying **_corrupt_char** to every character **c** in the sentence (via a generator expression).
 3. Joining the resulting characters back into a string.
 4. Collecting all corrupted sentences into a list and returning it.

5.3 Sample Output

```

Showing original vs. corrupted for first 5 sentences:
=====
Original:  O beautiful for patriot dream that sees beyond the years thine
alabaster cities gleam undimmed by human tears .
Corrupted: O Jeautiffl fzf patriot dream that 6ees beyond Dhe years thine
alabaster cities gleam undimmed by human tears .
-----
Original:  ( cf.
Corrupted: ( cf.
-----
Original:  The Village office of Western Union with George Towsley as manager
and telegrapher continued in Hard's drugstore until 1905 .
Corrupted: The Villag} office of WesterP Union with George Towsley as manager
)nd telegrapher continued in Hard's drugstore until 1905 .
-----
Original:  As if this was a signal , Poet abruptly began to thrash the water and
the quick movement slowly made them sink through the water .
Corrupted: As if this was a signal , Poet abruptly began to thrash the water and
the quic) movement slowly made tqem sinj thrVugh the water .
-----
Original:  Routine determinations were made for dissolved oxygen in the mixed
liquor and for oxygen uptake rates .
Corrupted: Routine deQerminati8ns were yade for disHolved oxygen in 'he mixed
liquor and for oxygen uptake rates .
-----

```

5.4 Results

- **Error rate:** Approximately 5% of non-space characters are altered, yielding a realistic level of noise.
- **Error distribution:** Uniform over letters, digits, and punctuation; no systematic bias.
- **Use in evaluation:** This corrupted corpus will serve as input to the spelling corrector. We can then compute Word Error Rate (WER) and Character Error Rate (CER) against the original `test_corpus` as ground truth.

6. Spelling Corrector Evaluation

6.1 Overview

We focus on two measures to assess the performance of the context aware spelling corrector, the average error rates:

- **Word Error Rate:** $WER = (S + D + I)/N$
- **Character Error Rate:** $CER = (S + D + I)/n$

where the symbols refer to string alignment after performing one of three transformations on the predicted text:

- ☐ S: number of substitutions
- ☐ D: number of deletions
- ☐ I: number of insertions
- ☐ N: number of words in reference sentence
- ☐ n: number of characters in reference sentence

Both are derived from the Levenshtein distance with values ranging typically between 0 and 1, where 0 indicates perfect alignment of the spelling corrector output with the reference (ground truth) and larger values indicate larger degrees of mismatch between the expected reference text and the corrected.

6.2 Implementation

The main object is the `SpellingCorrectionExperiment` class which is used for the evaluation of the performance of `spelling_corrector_function` on artificially corrupted data. It has the following components:

- 1) `__init__`: the initialiser of the spelling correction experiment evaluator.
 - a) `test_corpus`: the list of original, clean sentences (`List[str]`).
 - b) `spelling_corrector`: the spelling correction object capable of doing context-aware corrections on corrupted sentences.
 - c) `language_model`: a trained language model (here we considered only bigram and trigram models).
 - d) `error_prob`: the probability with which each non-space character will be replaced (default 0.05 = 5%).
 - e) `seed`: optional integer to seed the random number generator for reproducible outputs.
 - f) `beam_width`: an integer to control the depth of search of the corrector.
- 2) `run(self)`: the primary method of the class.
 - a) First, it generates the list of sentences `corrupted_sentences` as corrupted versions of the sentences via a `generator` instance of `ArtificialTestDataset` class with predetermined probability of character corruption.
 - b) Secondly, create the list `corrupted_tokenized` which contains the word tokens of each sentence from `corrupted_sentences` using the `word_tokenize()` function of NLTK. Each corrupted sentence is then passed through the provided spelling correction function.
 - c) Third, we create a list of `corrected_sentences` by applying the spelling corrector to every token of each corrupted sentence and concatenate them in a string to recreate the sentence.
 - d) Finally, we call the functions `wer_metric`, `cer_metric` from the `evaluate` library to compute the average WER and CER scores. The inputs are two lists: `corrected_sentences` and `reference_sentences`.

6.3 Results

The following table summarises the results for the trained bigram and trigram language models using 2000 test sentences for the evaluation process:

Model	CER	WER
Bigram	0.1002	0.3352
Trigram	0.0933	0.3336

Interpretation

The Word Error Rate for the bigram model is 33.52%, hence we can say that, on average, 33.5% of the words produced in the corrected sentences contain mistakes compared to the ground truth. This error rate in word level has a slight improvement if we adopt the trigram language model.

However, at a character level the spelling corrector has a more notable difference, with the bigram model achieving a Character Error Rate level of 10.02%, while the trigram model is reducing this to 9.33%. This means that on average we have a 10.02% of total characters misspelled for a corrected test sentence using a bigram model and 9.33% using a trigram model.

6.4 Conclusion

The results suggest that the trigram model improves the spelling corrector yielding lower values for both metrics. The character level precision is consistently better for both models. An important issue we can address is that the error rates align with the fact that the trigram model is context richer than the bigram, while the perplexity scores direct us to the opposite direction. From a bibliography inspection (reference: <https://www.cs.cmu.edu/~roni/papers/eval-metrics-bntuw-9802.pdf>) the perplexity score correlates with WER for in-domain data. As stated in the introduction, Brown corpus contains many categories of texts, therefore this relationship in our case is broken maybe due to the fact that we naively included the full corpus in our analysis.